

First, let us create the data-type 'complex' with two members a and b corresponding to the real and imaginary parts of a complex number.

```
CREATE TYPE complex AS (a float, b float);
```

Create a table with columns of data-type 'complex' and insert some rows in this table. By using the construct (x, y)::complex, we specify a 'complex' value with a = x and b = y.

```
CREATE TABLE complex_tab (val complex, val2 complex);
INSERT INTO complex_tab VALUES ((1, 1)::complex, (10, 2)::complex);
INSERT INTO complex_tab VALUES ((3, 4)::complex, (3, 3)::complex);
```

To add two 'complex' values using an addition (+) operator, we need to define the '+' operator. Let us define a function for the operator. The body of the function, which is written using procedural language PL/pgSQL, is encapsulated in '\$\$'. The arguments to the function can be accessed using the positional notations \$1 and \$2.

```
CREATE FUNCTION complex_add(complex, complex) RETURNS complex
AS $$
begin
    return ($1.a + $2.a, $1.b + $2.b)::complex;
end;
$$ LANGUAGE PLPGSQL;
```

Using the function 'complex\_add' we can create an operator '+' with both left and right arguments as 'complex'.

```
CREATE OPERATOR + (
    leftarg = complex,
    rightarg = complex,
    procedure = complex_add
);
```

Let us use the '+' operator to add both the columns of the 'complex\_tab' table using a SELECT query.

```
SELECT val + val2 as complex_add FROM complex_tab;
complex_add
-----
(11,3)
(6,7)
(2 rows)
```

We can use the same function 'complex\_add' to define aggregate 'sum' over 'complex' data-type. In the context of the earlier discussion on user-defined aggregates, we specify 'complex\_add' as the transition function (sfunc), which accepts arguments of 'complex' data-type (basetype)

and returns (stype) the value of 'complex' data-type. The transition result is initially 0 as shown by initcond.

```
CREATE AGGREGATE sum (
    sfunc = complex_add,
    basetype = complex,
    stype = complex,
    initcond = '(0,0)'
);
```

Now, let us use the aggregate we defined to calculate the sum of the 'val' column in all the rows of table 'complex\_tab'.

```
SELECT sum(val) FROM complex_tab;
sum
-----
(4,5)
(1 row)
```

Just like we defined addition (+) operator above, we can define various comparison operators like '<', '>', '=', etc. But for all of these comparison operators we will need a function to compare two values of data-type 'complex'. The function 'complex\_mag' defined below, returns the magnitude or absolute value of a complex number, which can be used to compare two complex values.

```
CREATE FUNCTION complex_mag(complex) RETURNS float AS $$
begin
    return sqrt($1.a * $1.a + $1.b * $1.b);
end;
$$ LANGUAGE PLPGSQL;
```

Using 'complex\_mag', we can define a function 'complex\_mag\_cmp', which returns -1, 0 or 1 if the first argument is less, equal or greater than the second argument, respectively.

```
CREATE FUNCTION complex_mag_cmp(complex, complex) RETURNS int
AS $$
declare
    mag1 float := complex_mag($1);
    mag2 float := complex_mag($2);
begin
    if (mag1 < mag2) then
        return -1;
    end if;
    if (mag1 > mag2) then
        return 1;
    end if;
    return 0;
end;
$$ LANGUAGE PLPGSQL;
```